OneScan

# A repo link proves nothing. OneScan checks it against the chain.

Portaldot Hacker House 2026, Node Operations & Tooling

onescan-portaldot.vercel.app · github.com/Techkeyy/onescan · youtu.be/JsZpk6Ul9JA

# Anyone can claim a repo. Nothing on V3 shows it is true.

### Judges

Score on repos and contract addresses, with no quick way to link the two.

### Integrators and users

Read the code on GitHub, then trust that it is what they are calling.

### Honest builders

Have no public way to prove their contract is the code they published.

Sourcify and Etherscan solve this on other chains. I found nothing doing it for Portaldot V3.

# Paste a repo and an address. Get a verdict anyone can recheck.

### Reads the chain

Running code, and the compiler version stored inside it.

### Pins the repo

One exact commit, so the answer cannot drift.

### Rebuilds it

Same compiler the contract was built with. Nothing from the repo is executed.

### Compares bytes

Anything weaker than equal is not a match.

No wallet. No account. A public proof page and a live README badge for every match.

# Run on real Hacker House contracts, through the public site

**40 of 43**

repo and contract pairs matched, across real teams

**32 of 132**

direct EVM deployments on V3 now have a public proof

**0**

false claims accepted in testing, with real repos against other teams' contracts

The other 3 got no verdict: their deployed code looks hand-written or their repo has no Solidity to build. OneScan says so instead of guessing.

Pairs found in public repos, run through the public site in November 2026 on a 1-CPU server. Every match has a public proof page.

The part that matters

# It refuses a false claim, and it never guesses.

### Does not match

Only after every setting ran, on the dependencies the repo pinned, with every source file built.

### Could not finish

Ran out of time, or a service was down. It says so, and never treats that as a failure of the repo.

# Why Portaldot

| What V3 brings | What OneScan does with it |
| --- | --- |
| A young EVM ecosystem | A trust layer early, while the contracts are still few enough to prove |
| Solidity through pallet-revive | Rebuilds against plain solc output, read from the V3 EVM RPC |
| EVM and PolkaVM on one chain | Tells them apart, and refuses to guess on code it cannot check yet |
| Costly deployments | Teams often strip metadata (8 of 43 here), so OneScan still checks those using the compiler the repo names |
| EVM code hashes on-chain | A live registry on the V3 testnet refuses any record unless the hash matches the running code. Five real projects are recorded so far: 0x2a64…dda1 |

The same method could work on other EVM chains. V3 is where it is missing today, and its multi-VM design is where it can grow.

# What a proof does not tell you

- A match says the code is what the repo shows. It is not an audit, and it does not say the code is safe.
- Solidity on the EVM side only. PolkaVM and ink! are not checked yet.
- Code that was never committed can never match.
- The on-chain record is OneScan’s statement. The real proof is that anyone can repeat the check with git, solc and one RPC call.

# Why it deserves support after the event

## Where it goes next

- Verify ink! and PolkaVM contracts
- A CI check that proves each release
- Show proofs inside explorers and wallets
- Mainnet, when teams move there

## How it could sustain itself

- Free public checks and proofs
- A paid API and CI for teams and integrators
- Verification as shared infrastructure for the ecosystem

No revenue yet, and adoption is still unmeasured. The model is a proposal. The working product and its results are real.

Try it: onescan-portaldot.vercel.app · Code: github.com/Techkeyy/onescan · Demo: youtu.be/JsZpk6Ul9JA